

SUR 2025/2026

Bc. Myron Kukhta

May 4, 2026

Contents

1	Introduction	2
2	Data	2
3	Image Binary Classification	2
3.1	Feature extraction	2
3.2	Image Classification Model	3
3.3	Training and Validation	3
4	Speech Binary Classification	4
4.1	Feature Extraction	4
4.2	Speech Classification Model	4
4.3	Training and Validation	5
5	Multy Modal Binary Classification	6
6	Implementation	6
7	Installation and startup	6
7.1	Data Preparation mode	7
7.2	Training	7
7.3	Production	8

1 Introduction

The goal of this project is to study the principles of machine learning using a real-world classification task with a limited amount of training data. As a result of this work, three binary classifiers were developed:

- an image-based binary classifier;
- a speech-based binary classifier;
- a multimodal binary classifier combining speech and image data.

Based on the input data, these models determine whether the target person is present or not.

2 Data

The dataset consists of 222 unique facial images in the `.png` format and 222 unique audio recordings of human speech in the `.wav` format. Data collection was organized into multiple sessions and conducted under different conditions, which introduced variability in lighting, environment, and recording quality.

The classes are unevenly distributed. Recordings of the target person contain 60 samples, while recordings of other individuals contain 384 samples. Such class imbalance may negatively affect the ability of the models to correctly estimate the class distribution.

When splitting the data into training and validation sets, I decided to follow our supervisor's recommendation and use cross-validation grouped by the combination of the user ID and the recording session. This approach allows for a more accurate assessment of the model's generalization ability under conditions of limited data diversity and prevents data from the same group from ending up in both the training and validation sets simultaneously. To distribute the data across groups, I used the `StratifiedGroupKFold` method with a 3-fold split.

3 Image Binary Classification

3.1 Feature extraction

Before feature extraction, the images were standardized by resizing them to 128×128 . In addition, to simplify data presentation, the images were converted to grayscale and their contrast was enhanced.

It was clear that using raw images for this task would be inefficient due to the high complexity of visual data structures. To obtain an optimal image representation, I was choosing between two popular feature extraction techniques HOG and LBP. Experiments conducted with various feature combinations revealed that the most effective approach was to use HOG features exclusively. Using LBP alone yielded poorer results, while combining HOG and LBP led to excessive noise and reduced representativeness.

HOG (Histogram of Oriented Gradients) is an image processing technique designed to extract information about object contours. The main idea of the method is to analyze the distribution of gradient directions within local regions of an image. This is especially important because object boundaries (in this case, facial features) are characterized by sharp gradient changes, while homogeneous regions tend to have more stable values.

In addition, a small amount of data augmentation was applied in order to increase dataset diversity despite the limited number of samples. The following augmentation techniques were used:

- random image rotation in the range from -10° to 10° ;

- random brightness and contrast adjustment;
- addition of random Gaussian noise.

During training, I observed a significant class imbalance problem. The number of images belonging to the target person was considerably smaller than the number of images belonging to other individuals. To partially address this issue, the training dataset was artificially expanded. For each image of the target person, three additional randomly augmented copies were generated and added to the training set instead of using only a single sample.

After extraction, the features are normalized using Z-score standardization and then transformed using PCA in order to reduce dimensionality and improve the informativeness of the feature space for the model.

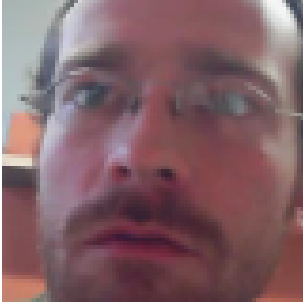


Figure 1: Example of original image



Figure 2: Example of image after preprocessing and augmentation



Figure 3: Example of HOG map

3.2 Image Classification Model

The classifier model is linear implementation of the SVM algorithm (LinearSVC). This choice was motivated by the fact that linear SVM models perform well on small datasets when high-quality features are available. Unlike nonlinear SVMs with kernel functions, LinearSVC trains faster and is less prone to overfitting on limited data. This suits my case, since after applying HOG and PCA, the features are expected to be more informative, partially decorrelated, and better separable by a linear boundary.

Additionally, the choice of SVM over, for example, logistic regression was driven by SVM's ability to maximize the margin. Given the small dataset and partial class imbalance, I assume that this approach would allow the model to generalize patterns better and reduce the likelihood of overfitting.

3.3 Training and Validation

During the training process, as mentioned previously, a cross-validation with supervised learning approach was used. Across several iterations, different combinations of preprocessing methods and model hyperparameters were evaluated in order to determine the optimal configuration.

The main issue encountered during training was the imbalance between the number of target and non-target samples. Due to this imbalance, the model tended to become biased toward one of the classes, which was particularly influenced by the value of the parameter C and the class weights. To reduce the impact of this imbalance, class weighting was applied, increasing the penalty for misclassification of the underrepresented class.

Further manual tuning of the parameter C and class weights was performed in order to stabilize the F1-score across different fold splits.

As a result, the final model achieved the following performance on the cross-validation splits (it is important to note that due to the class imbalance, the primary focus when evaluating model quality was on the F1-score metric, not accuracy):

Table 1: Cross-validation results for the image classifier

Fold	Class	Precision	Recall	F1-score
0	0	1.00	0.95	0.98
	1	0.77	1.00	0.87
1	0	0.98	0.98	0.98
	1	0.90	0.90	0.90
2	0	1.00	0.98	0.99
	1	0.91	1.00	0.95

Based on the results, we can say that the model performed well. It was able to establish a fairly robust decision boundary between classes, although in some cases it exhibits a slight bias toward the positive class, which is reflected in the precision value for class 1. Another positive outcome was that the artificial data augmentation applied did not lead to noticeable overfitting of the model.

4 Speech Binary Classification

4.1 Feature Extraction

Since human speech is a signal by nature, a logical and classical approach to its analysis is to transform the raw waveform into a frequency-based representation.

Before feature extraction, the raw audio signal was preprocessed in order to remove irrelevant parts and emphasize useful speech characteristics. First, all audio recordings were resampled to 16 kHz and converted to a mono channel. Then, silence was removed from the beginning and the end of each recording using an energy-based trimming method.

After that, a pre-emphasis filter was applied: $y[t] = x[t] - 0.95 x[t-1]$

This filter increases the relative contribution of high-frequency components. In speech signals, high-frequency regions often contain consonant-related information, which can be useful for distinguishing speakers.

The main feature extraction method used in this project was Mel-Frequency Cepstral Coefficients (MFCC). MFCCs provide a compact representation of the spectral envelope of the speech signal and are widely used in classical speech processing tasks. In this implementation, 16 MFCC coefficients were extracted using a 25 ms analysis window and a 10 ms hop length.

In addition to static MFCC features, delta coefficients were added in order to capture short-term temporal changes in speech. Delta-delta coefficients were tested as well, but were removed because they increased noise and made the classifier less stable.

Finally, the MFCC and delta features were concatenated into a single feature matrix, where each row represents one audio frame.

In this case, I was forced to decline from data augmentation, even though it was highly desirable, at least to mitigate the class imbalance problem described above. For some reason, the augmentation techniques I experimented with (noise injection, gain adjustment, and temporal shift) significantly degraded the classification performance.

4.2 Speech Classification Model

The classifier was implemented as a combination of two Gaussian Mixture Models (GMMs). The first model described the feature distribution of the target class, while the second model

represented the distribution of the non-target class. The final classification decision was made based on the log-likelihood ratio between the two models using a manually selected threshold.

The choice of GMMs can be justified by the assumption that a speech signal, after being transformed into MFCC features, can be represented as a mixture of several Gaussian distributions, where each component captures a certain latent factor influencing the structure of speech.

In addition, this architecture was chosen for research purposes, since I was interested in gaining practical experience with a generative model and studying its behavior in practice.

4.3 Training and Validation

A generative unsupervised approach combined with cross-validation was used to train the audio model. In each fold, two GMM models were trained separately: the first to model the distribution of features for the target class, and the second to model the distribution of features for the non-target class. Based on the obtained distributions of log-likelihood ratio, a decision threshold was manually selected, located between the mean values. You can view the obtained distributions below.

Table 2: Distribution of scores for audio cross-validation

Fold	Target Files				Non-Target Files			
	Mean	Std	Min	Max	Mean	Std	Min	Max
0	-0.894	0.562	-2.099	-0.210	-3.639	1.147	-6.131	-1.069
1	-1.577	0.348	-2.434	-1.199	-4.412	1.137	-7.489	-2.302
2	-1.919	0.455	-2.569	-1.363	-4.969	1.754	-9.528	-1.491

One of the first challenges was achieving sufficient separation between the class distributions. To improve separability, I had to abandon augmentation and select more appropriate MFCC parameters and the number of GMM components.

The next challenge was selecting a stable classification threshold. Despite threshold tuning, I was unable to simultaneously stabilize precision and recall across all cross-validation folds. The model exhibited a tendency toward over-classification, meaning it more often incorrectly classified non-target examples into the target class. I think, this problem is related to insufficiently informative feature extraction, as well as the lack of augmentation, which caused the model to learn the distribution of the target class poorly. Additionally, a more refined selection of the decision threshold may have been required.

Table 3: Cross-validation results for the audio classifier

Fold	Class	Precision	Recall	F1-score
0	0	1.00	0.89	0.94
	1	0.59	1.00	0.74
1	0	0.98	1.0	0.99
	1	1.0	0.90	0.95
2	0	0.95	0.98	0.97
	1	0.88	0.70	0.78

So, I consider the performance of the resulting model for the binary audio classification task to be satisfactory. Nevertheless, it can be assumed that during evaluation, the model will be prone to false positives and will often classify non-target classes as target.

5 Multy Modal Binary Classification

Unfortunately, due to time constraints, I was unable to implement a prototype of a multi-modal classifier that combines a person's image and voice for subsequent classification. But I can describe my proposed plan of action.

The model will follow the principles of feature extraction from previously implemented classifiers and consist of a combination of two independent feature spaces (visual and audio). To combine the results of both classifiers, I planned to use an additional logistic regression layer that would receive score values from each model as input. The logistic regression would then be trained to optimize the weights of these predictions to produce the final decision.

During training, I would also have used a supervised approach with cross-validation, but I would have split the data into image-voice pairs. My only concern was that the model might overemphasize the visual component and treat the audio more as noise due to the difference in their F1 scores.

6 Implementation

The project was implemented in Python using an object-oriented approach and standard machine learning frameworks, including scikit-learn, as well as libraries for mathematical computations (NumPy, Pandas), images (OpenCV), and audio (Librosa). Additionally, system utilities and auxiliary libraries were used to organize the pipeline.

Below you can review the project structure:

```
project/
|-- configs/           # example of projects configs
|-- src/               # source code of project
|   |-- data/          # module for working with data
|   |-- model/          # implementations of models and its components
|   |-- production/     # module for production pipelines
|   |-- train/          # module for train pipelines
|-- models_output/
|   |-- audio_out.txt   # result of audio eval
|   |-- image_out.txt   # result of image eval
|-- train_out/         # result of model training with logs and .pkl
|-- main.py            # entry point of project
|-- README.md          # brief guide to launching the project
|-- requirements.txt    # project dependencies
```

In addition to the source code, I have provided examples of configuration files (configs/), model training results (train_out/), and model testing results on the eval dataset (eval/).

7 Installation and startup

This project is primarily designed for a Linux environment. To prepare the project, it is necessary to create a Python virtual environment:

```
python3 -m venv venv
```

After that, the virtual environment should be activated:

```
source venv/bin/activate
```

Next, install all required dependencies from the `requirements.txt` file:

```
pip install -r requirements.txt
```

After completing these steps, the project will be ready for use. The system can then be operated in several modes:

- data preparation,
- training,
- production

For details info use:

```
python3 main.py --help
```

7.1 Data Preparation mode

This mode is intended for generating a metadata map of the dataset and currently supports only `.wav` and `.png` files. After, the program automatically separates the data by modality (image and audio) and generates `CSV` files containing a list of all detected samples along with their corresponding labels, when such labels are available.

When the extended mode is enabled, the system additionally performs cross-validation splitting into 3 groups. Separate `CSV` files are generated for each fold.

This mode can be launched using the following command:

```
python3 main.py -d [PATH_TO_CONFIG]
```

The command expects a configuration file in `JSON` format. Example configuration files can be found in the `configs/` directory.

Table 4: Configuration schema description

Config Parameter	Type	Description
source_data_dirs	list of string	Paths to all the directories where your data is stored.
output_data_dir	string	The path to the directory where the prepared metadata will be saved.
trg_person_id	string	Specify the target person's ID for classification in the file name for labeling. If no match is found, the file will be assigned the default class 0. For example: <code>m431</code> .
with_folds	boolean	Should we split the data into folders for cross-validation?.

7.2 Training

This mode assumes that the metadata preparation step described above has already been completed before execution.

The mode launches training of the selected model according to the provided configuration file. Training results and evaluation metrics are displayed in the command-line output.

The system supports running cross-validation without saving model weights, as well as training on the full dataset with subsequent saving of the trained model weights. Additionally, training logs can optionally be exported to a `.txt` file for further analysis.

This mode can be launched using the following command:

```
python3 main.py -t [PATH_TO_CONFIG]
```

The command expects a configuration file in JSON format. Example configuration files can be found in the `configs/` directory.

Table 5: Configuration schema description

Config Parameter	Type	Description
model	string	Type of classifier: "audio" vs "image".
total_data_csv_path	string	The path to the CSV file with the total samples of actual type of data generated in <code>-d</code> mode.
folds_dir_path	string	The path to the directory with all folds separations generated in <code>-d</code> mode.
out	string	The path to the directory where will be stored model <code>.pkl</code> and logs.
model_name	string	ID for actual model running, only for logs and better separability.
is_full_train	boolean	Enables final training on the full dataset.
is_save_validation_log	boolean	Enables saving cross-validation logs.

7.3 Production

This mode assumes that the metadata preparation step described above has already been completed before execution, separation on folds can be ignored.

This mode runs the classification of the input data using the selected model and saves the result in the same format that the project expected for the classification report of the evalv data.

This mode can be launched using the following command:

```
python3 main.py -p [PATH_TO_CONFIG]
```

The command expects a configuration file in JSON format. Example configuration files can be found in the `configs/` directory. You only need to specify the paths to the model weights in configuration if you select this classifier. Otherwise, the path can be left blank.

Table 6: Configuration schema description

Config Parameter	Type	Description
model	string	Type of classifier: "audio" vs "image".
data_path	string	The path to the CSV file with the total samples of actual type of data generated in <code>-d</code> mode.
audio_target_model_path	string	The path to the <code>.pkl</code> file of weights of GMM for target audio.
audio_non_target_model_path	string	The path to the <code>.pkl</code> file of weights of GMM for nontarget audio.
image_model_path	string	The path to the <code>.pkl</code> file of weights of image classifier.
classification_out_path	string	Output path for classification report <code>.txt</code> file.